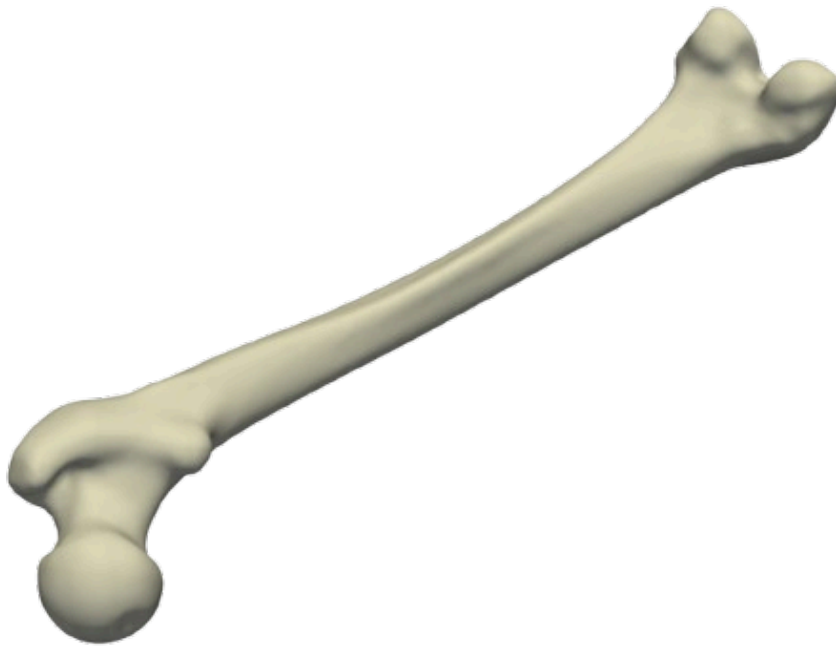


Statistical and Neural approaches in 3D Shape Modeling



Grenoble INP : ENSIMAG
Master's in Applied Mathematics
Modeling Project in C++ - Marek Bucki
Spring 2026

Martin Lainé, Basile Mouret, Timothé Boyer, Malik Hacini

30.01.2026 -

Abstract

For this project, we developed a comprehensive statistical shape analysis system for femur bones. Our approach relies on two complementary methodological pillars: Principal Component Analysis (PCA), enabling linear dimensionality reduction and identification of the main variation modes, and artificial neural networks, providing the capability to capture non-linear relationships in the data. Each femur in our dataset is represented as a 3D mesh comprising 18,291 vertices, resulting in a vector of dimension 54,873 in the shape space. The main objective of this work is to build a statistical shape model capable of reducing the dimensionality of the data using an autoencoder built with neural networks, generating new anatomically plausible femur shapes. However, during our research, we were unable to guarantee biological consistency with regard to the anatomical consistency of the femurs, which is possible thanks to variation. This led us to a third avenue of research concerning the LDDMM framework.

Introduction

Here is the report on our work on “Statistical and Neural Approaches to 3D Femur Modeling.” In the field of medical imaging, understanding the variations in the shapes of different parts of the human body is a crucial issue. During our work, we focused on human femurs and asked ourselves the following question: “How can we model and compress the complex 3D shape of a femur and explore the possible variations?”. To answer this question, we worked with a dataset of 24 femur scans represented as a dense point cloud. Our goal was to reduce this high-dimensional data into a compact set of parameters that preserve the anatomical structure. We explored three main approaches to achieve this goal: Principal Component Analysis (PCA), Neural Networks, and Large Deformation Diffeomorphic Metric Mapping (LDDMM). Each of these methods offers unique advantages and challenges in capturing the intricate variations in femur shapes. In this report, we present our methodology, results, and insights gained from applying these techniques to femur shape modeling.

1 | Motivation

Modeling the 3D shape of the human femur is a central challenge in medical image analysis. The femur exhibits both global similarities and subtle anatomical variations across individuals, which are crucial for applications such as surgical planning, implant design, and pathology detection. However, each femur in our dataset is represented by a dense 3D mesh with over 18,000 vertices, resulting in extremely high-dimensional data (54,873 dimensions per shape).

The main challenge is to find a compact, informative representation of this complex shape space that preserves anatomical variability while enabling efficient analysis and generation of new, plausible femur shapes. Traditional linear methods like PCA can capture the main modes of variation, but may miss non-linear relationships that are important for biological realism. Neural networks, on the other hand, offer the potential to learn these non-linearities, but require careful design and validation to ensure anatomical plausibility.

In this project, we focus on the problem of dimensionality reduction for femur shapes: how can we compress such high-dimensional anatomical data into a small set of parameters, without losing essential information? Addressing this question is key to advancing statistical shape modeling and enabling new applications in personalized medicine.

2 | PCA

2.1 Linear Principal Component Analysis (PCA)

Our first approach to understand the shape variability of femur bones is through Linear Principal Component Analysis (PCA).

2.1.1 Principle

It's clear that in general, femur bones have a similar structure, but they can vary in size, curvature, and other shape characteristics. PCA is a statistical technique that helps us to identify and quantify these variations. It's a change of basis that aims to find the directions (principal components) in which the data varies the most. By projecting the data onto these principal components, we can reduce the dimensionality of the dataset while retaining most of the variance.



Figure 1 - 3D visualisation of a femur. We recognize a general shape

2.1.2 Change of Basis

To perform PCA, we start with a dataset of femur bone shapes represented as high-dimensional vectors. We note our N femurs $S_i \in \mathbb{R}^{3P}$ the shape vector of the i^{th} femur, where P is the number of points used to represent the shape. The first step is to compute the mean shape vector $\bar{S}$:

$$S = \frac{1}{N} \sum_{i=1}^N S_i \quad (1)$$

We then center the data by subtracting the mean shape from each femur shape vector and compute the covariance matrix C of the centered data:

$$C = \frac{1}{N-1} \sum_{i=1}^N (S_i - \bar{S})(S_i - \bar{S})^T \quad (2)$$

Our goal is to find a change of basis for our centered vector that all our data are uncorrelated. That means we want to find a basis where the covariance matrix is diagonal. This is achieved by finding the eigenvalues and eigenvectors of the covariance matrix C . The eigenvectors represent the directions of maximum variance (principal components), and the corresponding eigenvalues indicate the amount of variance captured by each principal component.

2.1.3 Dimensionality Reduction

Once we have the principal components, we can project the original femur shape vectors onto a lower-dimensional subspace spanned by the top K principal components. This is done by selecting the K eigenvectors v_k corresponding to the largest eigenvalues λ_k .

Any femur instance S_i in the dataset can be approximated as the mean shape plus a weighted sum of the principal components.

$$S_i \approx \bar{S} + \sum_{k=1}^K w_k v_k \quad (3)$$

where w_k correspond to the standard déviations along each principal component direction.

2.1.4 Results, visualisation and limitations

Doing the pca on our femur dataset, allow us to reduce the dimensionality from 54873 (3×18291) to 10 while still capturing a significant amount of the variance in the data.

However, PCA has its limitations. It assumes that the data lies on a linear subspace, which may not always be the case for complex shapes like femur bones. Additionally, PCA is sensitive to outliers and may not capture non-linear relationships in the data. To address these limitations, we also explored non-linear dimensionality reduction techniques, such as Neural Networks.

3 | Autoencoder

3.1 Neural Networks

Neural Networks are a class of machine learning models inspired by the structure and function of biological neural networks. They are particularly well-suited for modeling complex, non-linear relationships in data. In this section, we describe the architecture and training process of the neural network implemented for our shape analysis task.

3.1.1 Modelisation of Neurons

A single artificial neuron receives an input x of size n , $x \in \mathbb{R}^n$. Each component is associated with a weight. The neuron computes a weighted sum of these inputs and adds a bias term. Mathematically, this can be expressed as:

$$f(x) = b + \sum_{k=1}^n w_k x_k \quad x = (x_1 \dots x_n) \quad (4)$$

However, this result is just a linear combination of the inputs. To introduce non-linearity into the model, an activation function is applied to this weighted sum.

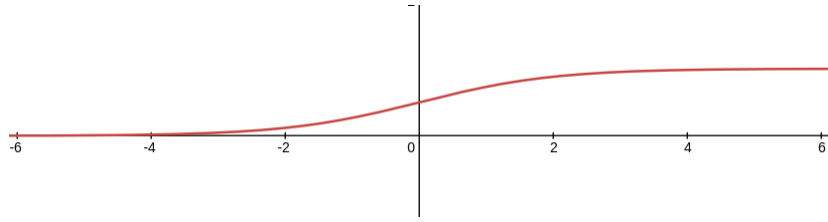


Figure 2 - Sigmoid activation function $\Phi(x) = \frac{1}{1+e^{-x}}$

Remark: The sigmoid function is one of many activation functions used in neural networks. Others include ReLU (Rectified Linear Unit), Tanh, or Softmax, each with its own advantages and applications.

Finally, the output of the neuron is given by the activation function applied to the weighted sum:

$$(\Phi \circ f)(x) = \Phi(f(x)) \quad (5)$$

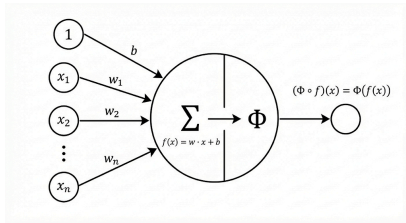


Figure 3 - Structure of an Artificial Neuron

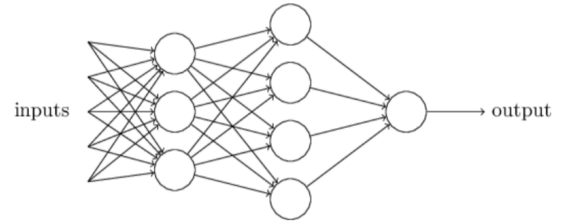


Figure 4 - Structure of a Neural Network

3.1.2 Layers and Network Architecture

Artificial neurons are organized into layers to form a neural network. A typical feedforward neural network consists of an input layer, one or more hidden layers, and an output layer. Each layer contains multiple neurons, and the output of one layer serves as the input to the next layer. However, the weights and biases of the neurons are initially set to random values and need to be optimized through a training process.

3.1.3 Backpropagation and Training

Training a neural network involves adjusting the weights and biases to minimize the difference between the predicted outputs and the actual target values. This is typically done using a method called backpropagation combined with an optimization algorithm like gradient descent.

The training process consists of the following steps:

1. **Forward Pass:** The input data is passed through the network layer by layer to compute the output predictions.
2. **Loss Calculation:** The loss function quantifies the difference between the predicted outputs and the actual target values.
3. **Backward Pass:** The gradients of the loss function with respect to each weight and bias are computed using the chain rule of calculus. This process is known as backpropagation.
4. **Weight Update:** The weights and biases are updated using the computed gradients and a learning rate, which determines the step size for each update.

3.1.4 Neural Network Implementation

For this project, we implemented a Neural Network from scratch in **C++**. This enabled us a deep understanding of the method and the underlying computations. We started by creating a **Vector** and **Matrix2D** class for which we implemented all the necessary linear algebra methods for a neural network. We used the **Eigen** library to store the data as we were only interested in the mathematical implementation. We then created a **Neural Network** class defining fully connected neural networks with specific activation and loss functions. This class also has the methods to do a forward pass of the neural network and train it using backpropagation.

3.1.5 Autoencoder Structure

In order to reduce the dimensionality and to discover underlying relationships in the data we used an Autoencoder structure. This is a type of fully connected neural network which tries to regenerate the input data while passing through a very small layer. This layer called the latent space acts as a bottleneck for information transfer between the encoder, the part before the latent space, and the decoder, the part after it. Our input and output layers consist of 54873 neurons as we had 18291 three dimensional points for each femur. By empirical testing, we chose a latent space of size 10 as it seemed enough to capture the main component of the dataset. As we used our own implementation of neural networks, which isn't as efficient a state of the art libraries, we had to aggressively compress each layer, going from the full 54873 down to 256, 32 and finally 10 neurons for the latent space. Furthermore, in order to get faster training, we did some preprocessing by subtracting the mean femur before feeding them to the autoencoder.

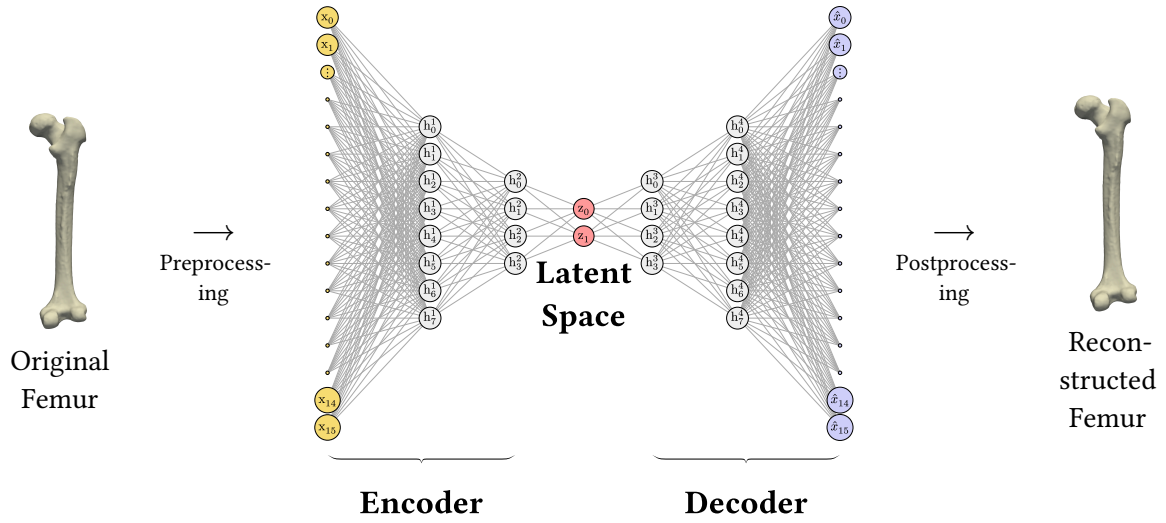


Figure 5 - Autoencoder pipeline for femur mesh reconstruction

We used the Mean Squared error for the loss function as all the point where corresponding to one another. We also tried different activation functions, and got the best results with LeakyReLU and tanh.

As we implemented the neural network and linear algebra ourselves, the training took a long time. We then focused on increasing performance.

3.2 Multi-threading

Since training the neural network is quite slow, we used multi-threading to speed up the process. We focused on parallelizing the matrix-vector multiplication (MatVec) operations, as these are computationally intensive and benefit the most from parallel execution.

3.2.1 With `std::threads`

Our first approach was to use `std::threads` from the C++ standard library.

3.2.1.1 Naive method

At first, we naively tried to create a thread for each operation that could be parallelized. However, the overhead from creating so many threads was far greater than any time saved. For example, for the largest matrix in our network ($54873 \times 512 = 28,094,976$ parameters), we would have created 28,094,976 threads, which is obviously not feasible. With this approach, our neural network was actually slower than the single-threaded version.

3.2.1.2 Improved approach

We then decided to split the computation among a fixed number of threads (depending on the number of CPU cores, typically 8 or 16). Each thread computes a portion of the result vector: the first thread computes the first $\frac{\text{result.size}()}{\text{num_threads}}$ elements, the second thread the next $\frac{\text{result.size}()}{\text{num_threads}}$ elements, and so on.

This approach worked much better, and we observed a significant speedup in the training time of our neural network. However, there was still some overhead due to thread creation and destruction at each MatVec operation.

3.2.1.3 Thread pool

As noted in Section 3.2.1.2, we were still creating and destroying too many threads. The best solution would be to implement a thread pool, where a fixed number of threads are created at the start of the program and reused for multiple MatVec operations. Unfortunately, due to time constraints, we did not implement this method, which could have further improved the performance of our neural network training.

3.2.2 OpenMP

A second approach was to use OpenMP, a popular API for parallel programming (see [1]). It provides a simple and efficient way to parallelize code, as it is close to sequential code and automatically manages thread creation and workload distribution.

3.2.3 Performance Comparison

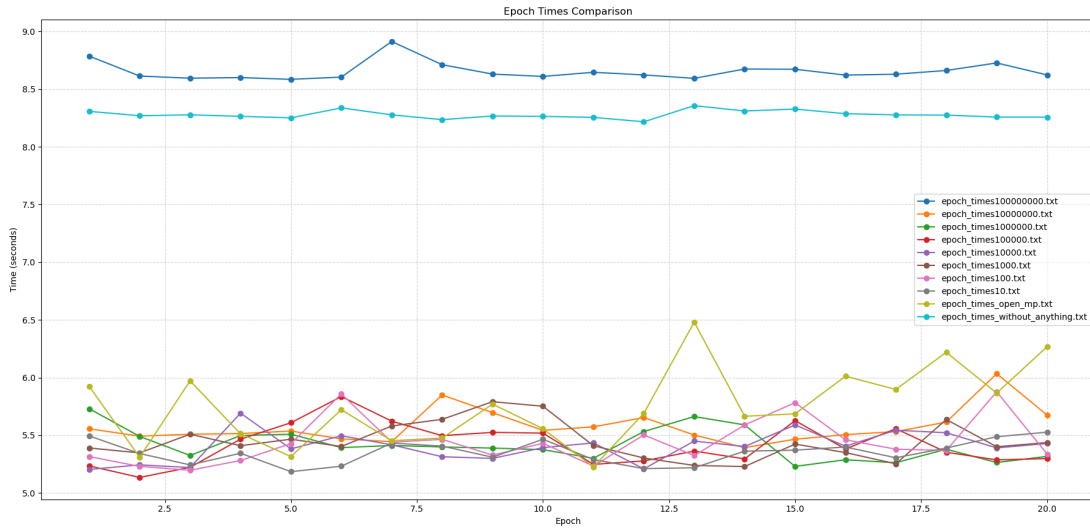


Figure 6 - Performance comparison between single-threaded and multi-threaded implementations.

Note: The number at the end of epoch_times corresponds to the threshold parameter, which is the number of parameters in the weight matrix above which we use multithreading.

As shown in Figure 6, using multi-threading significantly reduces the time required to train the neural network: we gain about 3 seconds per epoch, which is quite significant since we train our neural network for hundreds of epochs.

Even though we achieved better results with this multi-threaded approach, as mentioned in Section 3.2.1.3, we could have obtained even greater improvements by implementing a thread pool with `std::threads`.

Overall, the multi-threaded implementation shows a clear advantage over the single-threaded version, especially as the size of the dataset increases. This demonstrates the effectiveness of parallelizing computationally intensive operations in our neural network training process.

Remark: We observed that the time taken by our neural network to train with OpenMP multi-threading or with our `std::threads` implementation is quite similar. The OpenMP method likely uses an algorithm similar to the one we implemented with `std::threads`.

Remark 2: As expected, with the threshold parameter set to 100,000,000, all computations are single-threaded (since the largest matrix in the neural network contains 28,094,976 parameters), so the training time matches that of the single-threaded implementation.

3.3 Optimizing Memory Allocations

The main bottleneck of our Neural Network implementation is memory bandwidth as most of the training and inference time is lost waiting for data to be loaded from memory. This meant we had to optimize the memory allocations of our computations.

In order to increase our performance, we adopted a data oriented design programming approach by increasing memory cache efficiency. As matrices are the largest data structures of our programs, we had to understand how they were stored. There are two main convention, in **C** multidimensional arrays are row-major, so they are stored from left to right then up to down. On the contrary, in **Fortran** arrays are stored in columns-major order.

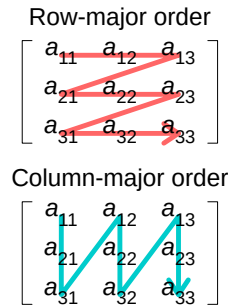


Figure 7 - Illustration of row- and column-major order by CMG Lee.

For compatibility reasons with linear algebra libraries in **Fortran** like **BLAS** and **LAPACK**, **Eigen** uses the column-major order by default. This means elements from a same columns are near one another in memory. So when we access elements of a matrix, we should iterate over elements from the same columns as they will already be loaded in the cache, resulting in faster load times.

```
for (size_t j=0; j < m_rows; ++j){
  for (size_t i=0; i < m_cols; ++i){
    // operation
  }
}
```

→

```
for (size_t j=0; j < m_cols; ++j){
  for (size_t i=0; i < m_rows; ++i){
    // operation
  }
}
```

Listing 1 - Exchanging iteration order results in 2x faster model training by reducing cache misses

We then reduced the amount of memory that was created and destroyed during the neural network methods by preallocating matrices and vectors. These are placeholders in memory created before a function call and that are reused, thus removing the overhead of allocating and destroying memory. The allocation and destruction overhead.

Finally we combined some linear algebra operations to remove temporary matrices. For example when computing $y = A^t \cdot x$ during the backpropagation, we were creating a temporary transpose matrix :

```
Matrix2D<T> W_transpose = m_weights[layer + 1].transpose();  
Vector<T> weightedDelta = W_transpose * deltas[lastLayer - layer - 1];
```

In order to eliminate this, we created a new method on the **Matrix2D** class that combines both operations efficiently :

```
Vector<T> multiplyTranspose(const Vector<T>& vec) const {  
    Vector<T> result(m_cols);  
  
    for (size_t j = 0; j < m_cols; ++j) {  
        T dot = 0;  
        for (size_t i = 0; i < m_rows; ++i) {  
            dot += m_data(i, j) * vec(i);  
        }  
        result(j) = dot;  
    }  
    return result;  
}
```

In total, optimizing our memory usage made our training epochs eight times faster.

3.4 Tools to debug and assess the quality of the reconstruction

To help us debug and assess the quality of our neural network, we developed several Python scripts using the PyVista library [2]:

3.4.1 visuFemur.py

This was the first script we implemented. It allows us to visualize a femur mesh from a .OBJ file. We initially used it to understand our data, and later to debug and visualize femur meshes generated by the neural network.

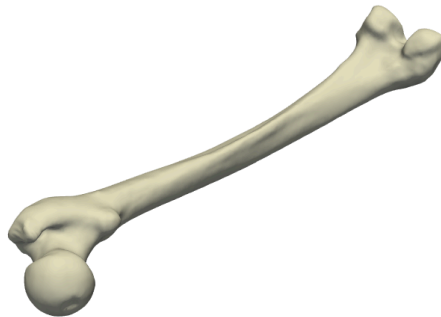


Figure 8 - Visualization of a femur mesh using **visuFemur.py**

3.4.2 compare_femur.py

We also implemented a script to compare two femur meshes, displaying a heat map of the differences (using the $\|\dots\|_1$ norm). We mainly used it to compare original and reconstructed femurs, to debug our neural network and assess its reconstruction quality.

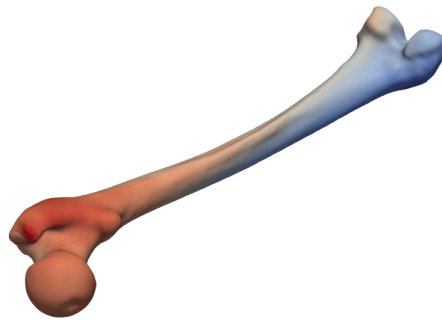


Figure 9 - Heat map of differences between two femur meshes using `compare_femur.py`

3.4.3 `latent_explorer.py`

Finally, we developed a script to explore the latent space of the neural network. It allows us to modify the 10 latent variables using sliders and see the corresponding reconstructed femur mesh in real time.

This tool is very useful for debugging and testing the neural network, especially for understanding how the latent space is structured and how the different latent variables affect the reconstructed femur shape. It can also be used to generate new femur shapes by exploring the latent space.

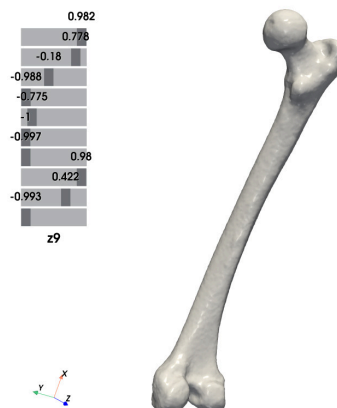


Figure 10 - Latent space explorer using `latent_explorer.py`

With these sliders, we can generate a wide variety of femur shapes by moving the 10 latent variables. This was very useful for testing the generative capabilities of our neural network and for checking if it could generate realistic femur shapes.

However, by moving the sliders, we also noticed that some generated femurs were not realistic. That's why we decided to perform a PCA on the latent space, to see if we could identify directions that lead to realistic femurs (see Section 3.5).

3.5 Encoder results

The goal of the encoder is to reduce the dimensionality of the femur shapes from 54,873 to 10.

We decide to plot the data in the latent space using only these three components. To see if there is some structure in the data.

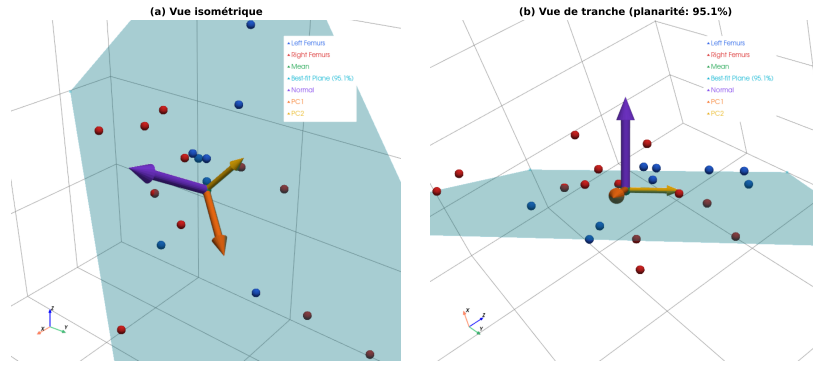


Figure 11 - Latent space representation using the first three principal components. We draw one plane approximating the data distribution.

With this plot, we can see that the data is not uniformly distributed in the latent space. We decide to do a PCA on the latent space to see if we can focus on fewer dimensions.

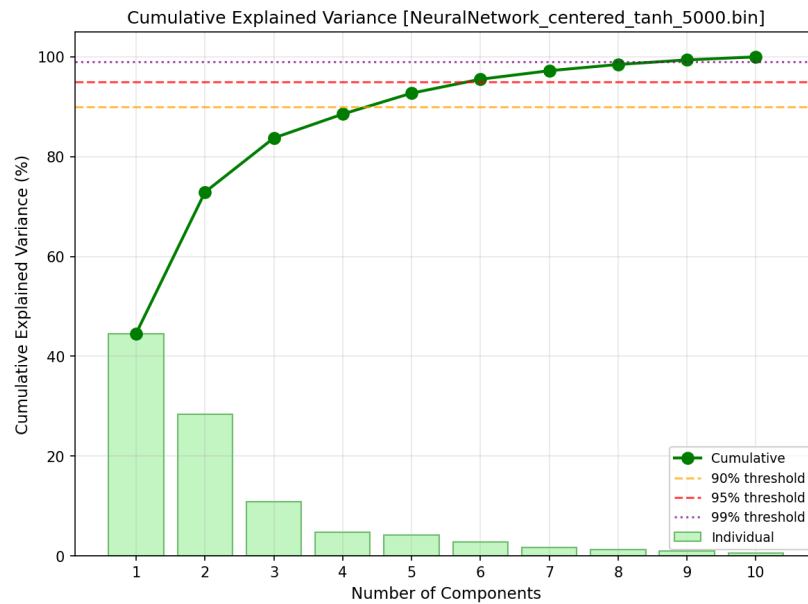


Figure 12 - Cumulative variance explained by PCA on the latent space

With this plot, we can see that the first six principal components explain almost 90% of the variance in the latent space. This means that we can reduce the dimensionality of the latent space from 10 to 6 without losing much information.

4 | LDDMM

5 | Discussion

We compared PCA and neural networks to model the 3D variability of femur shapes. PCA efficiently reduces dimensionality and captures the main linear modes, but struggles with non-linear anatom-

ical variations. The neural network autoencoder overcomes this by learning non-linear features, leading to better reconstructions and the ability to generate new, plausible femurs.

Exploring the latent space of the neural network, we found that most of the variance is explained by a few directions: the first six principal components account for almost 90% of the variability. This shows that the network compresses the essential information into a compact, structured subspace.

Our Python tools for visualization, mesh comparison, and latent space exploration were crucial for debugging and evaluating the models. They revealed that, while the neural network can generate a wide range of shapes, some are not anatomically realistic—highlighting the need for more constraints or regularization.

In short, combining PCA and neural networks gives a flexible and powerful framework for shape modeling. The neural network’s ability to capture non-linear variations and generate new shapes is a significant advantage, but ensuring anatomical plausibility remains a challenge for future work.

6 Conclusion and future works

In this work, we showed that both PCA and neural networks are effective for reducing the dimensionality of femur shapes. PCA provides a compact linear description of the main modes of variation, while the neural network captures non-linear features and is able to generate new, plausible femur shapes. Analysis of the latent space confirms that the essential information can be represented in a low-dimensional, structured way.

To improve our results, several directions are possible.

One possible improvement is to augment the dataset using PCA-based generation. While this does not add new information, since it only creates linear combinations of existing data, it can still help the neural network generalize better.

Another direction is to enhance the neural network architecture and training process, for example by increasing the depth of the network. This could allow the network to learn more complex features, but would require careful tuning to avoid overfitting and also the training process will be longer.

We can also improve the training time of the neural network by implementing more advanced multi-threading techniques, such as a thread pool (see Section 3.2.1.3).

The network could also be used to do clustering on healthy and unhealthy femurs, by training it on a larger dataset containing both types of femurs. This would allow us to better understand the differences between these two categories and potentially identify pathological shapes.

Finally, exploring more advanced generative models, such as Variational Autoencoders, could provide better control over the generation process and improve the realism of the generated femur shapes. These models can learn more complex distributions and might help in generating anatomically plausible shapes.

Glossary

Bibliography

- [1] “OpenMP.” [Online]. Available: <https://www.openmp.org/>
- [2] “PyVista.” [Online]. Available: <https://docs.pyvista.org/index.html>